CRAB: A NOVEL CLUSTERING SCORE USING CLUSTERING WITH RIVALS AND BUDDIES FOR UNSUPERVISED LEARNING

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Statistics

by

Allen Choi

June 2026

© 2026

Allen Choi

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE: cRab: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning

AUTHOR: Allen Choi

DATE SUBMITTED: June 2026

COMMITTEE CHAIR: Kelly Bodwin, Ph.D.
Thesis Advisor

COMMITTEE MEMBER: Alexander Dekhtyar, Ph.D.
Professor of Statistics

COMMITTEE MEMBER: Kevin Ross, Ph.D.
Professor of Statistics

ABSTRACT

cRab: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning

Allen Choi

We present a novel clustering score approach relying on a method that classifies pairs of points as either ”rivals” or ”buddies”, constructing a score for a given number of clusters. We then present results demonstrating the properties of this metric on datasets with no clusters.

Keywords: Clustering. Unsupervised Clustering. Score. Machine Learning.

ACKNOWLEDGMENTS

I'd like to first thank my amazing Mom and Dad for providing me so much emotional support over these past half a decade at Cal Poly. Without them, I don't know how I would have gotten through college.

But without a doubt, in regards to my statistical knowledge and skills I have no one else to thank but the amazing Statistics faculty at Cal Poly, who have always been patient, understanding, and kind.

Also a thank you to Andrew Kerr for providing a quarto template: https://github.com/Wander03/CalPolySTAT-thesis-template.

TABLE OF CONTENTS

Page

LIST OF TABLES vii

LIST OF FIGURES viii

CHAPTER

1 The Problem and What's been Tried 1

1.1 Motivation 1

1.2 Literature Review 5

2 Details 6

2.1 Calculated 6

2.2 Performance on Simulated Data 6

2.3 Sensitivity 6

2.4 Known Data 6

2.5 Performance on penguins dataset 6

3 Computational Details 7

3.1 Matrix Stacking 7

3.2 Algorithm Construction 7

3.3 Score Construction 7

3.4 Distribution of Ones and Neg Ones 9

3.5 Speeed and Memory Scaling 9

4 Null Distribution 10

4.1 Null Distribution of Scores 10

4.2 Null Distribution over Resamples 11

5 Conclusion 12

REFERENCES 13

APPENDICES

A Resample Matrix 14
B Mean Matrix 18
C Metric Calculation 19
D Point Contribution Function 20
E Get Point Vector Helper 21

LIST OF TABLES

Table Page

4.1 Variances for 20% Resample Calculation 11

LIST OF FIGURES

Figure Page

4.1 Distribution of Scores on a Random p-Dimensional Normal Distribution, Faceted by Resample % 10

Chapter 1

THE PROBLEM AND WHAT'S BEEN TRIED

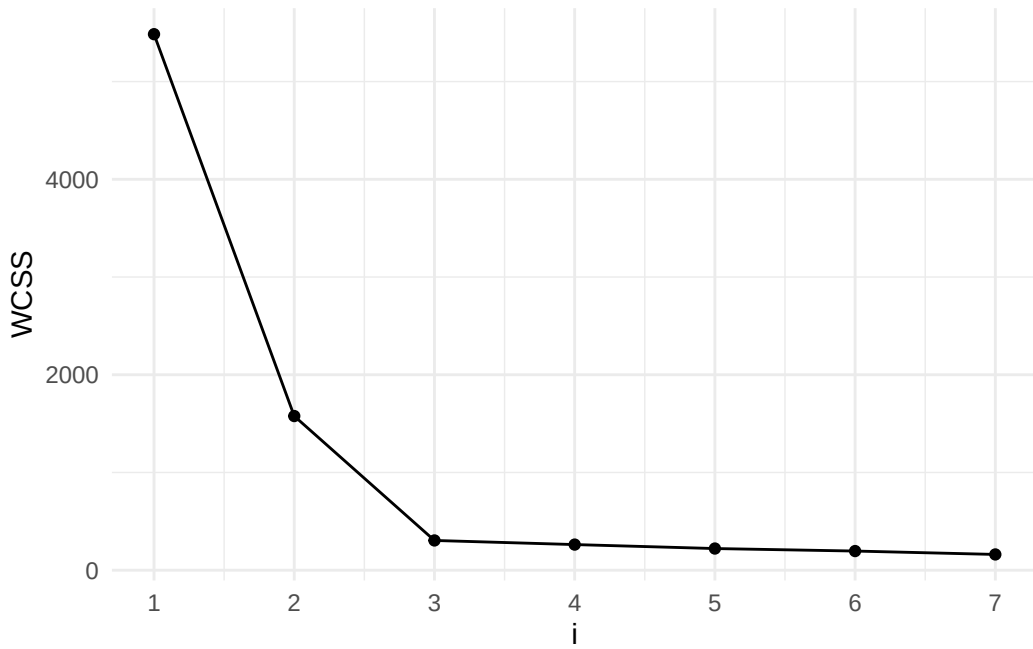
1.1 Motivation

Unsupervised clustering algorithms such as K-Means and hierarchical clustering are commonly used today when the number of clusters in a given dataset is not known beforehand, differentiating it from supervised clustering. Kodinariya (2013) presents the most common approaches used, such as the elbow method and Silhouette score. However, even with these scores there are often disagreements and caveats. For example, the elbow method sometimes presents itself as having no elbow, and the definition of an "elbow" is sometimes ambiguous (we need a source for this later). There are also cases where the scores don't necessarily agree, causing confusion in what score to believe.

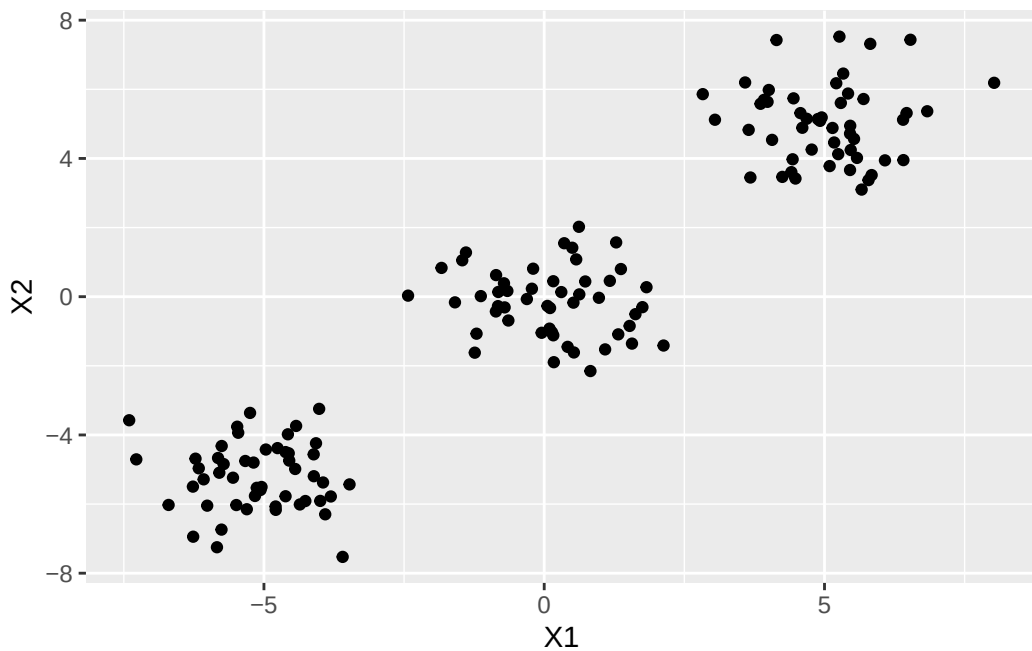
```
data <- data.frame(rbind(
  cbind(rnorm(50, -5), rnorm(50, -5)),
  cbind(rnorm(50, 0), rnorm(50, 0)),
  cbind(rnorm(50, 5), rnorm(50, 5))))

res <- lapply(c(1:7), function(i) {
  WCSS <- kmeans(data,
                 centers = i, nstart = 10)$tot.withinss
  return(data.frame(i = as.vector(i),
                    WCSS = as.vector(WCSS)))
})
bind_rows(res) |>
```

```r
  ggplot(aes(x = i, y = WCSS)) +
  geom_point() +
  geom_line() +
  theme_minimal() +
  scale_x_continuous(breaks = 1:7)
```



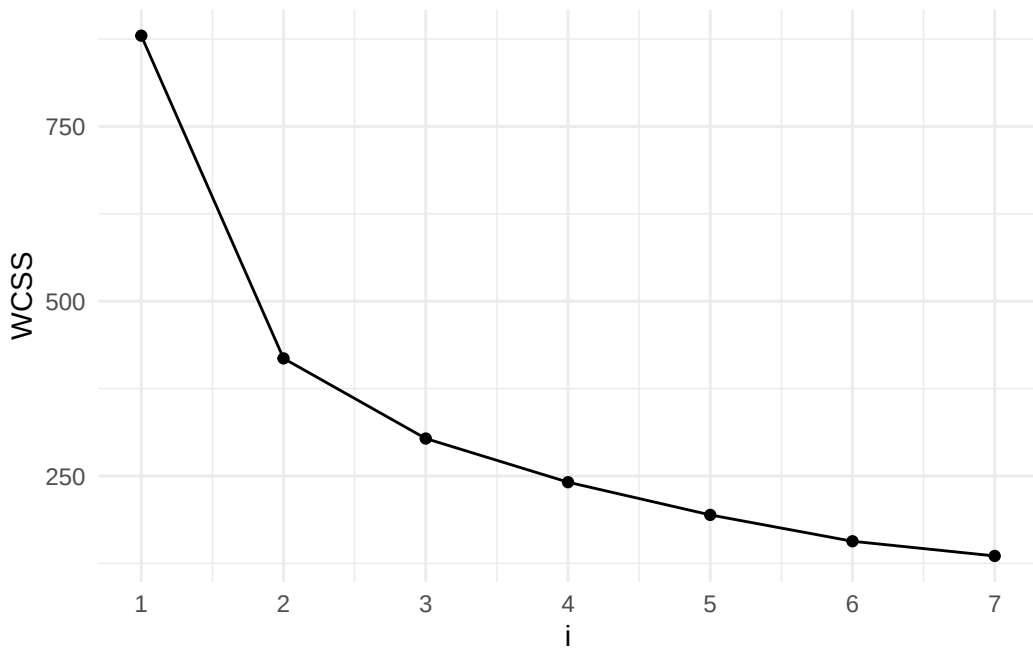

```r
ggplot(data = data, aes(x = X1, y = X2)) +
  geom_point()
```



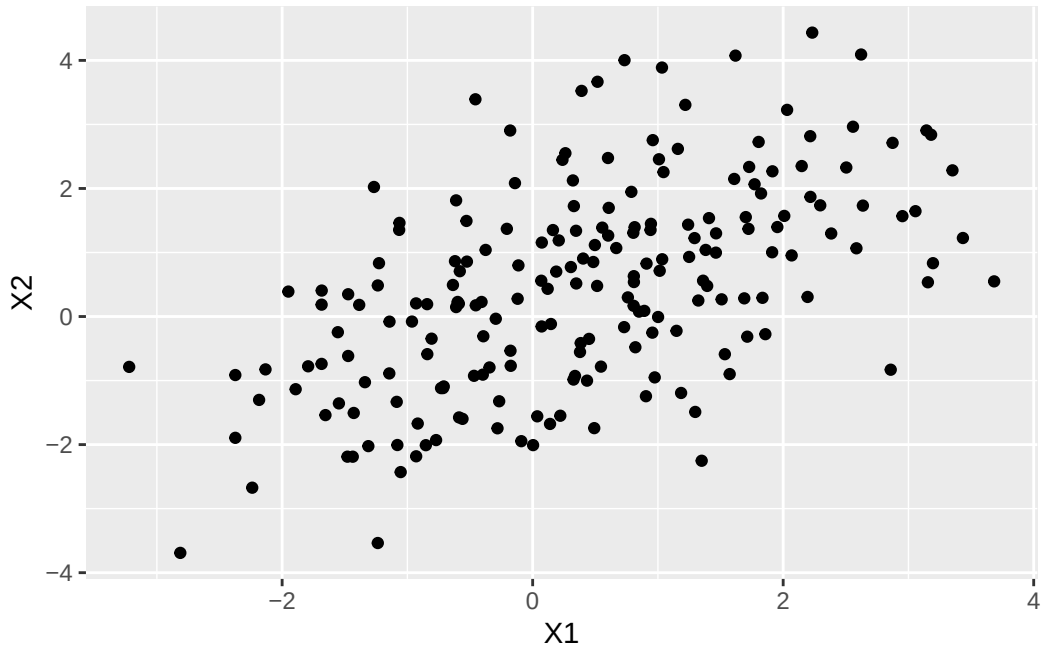

```
data <- data.frame(rbind(
  cbind(rnorm(50, -1), rnorm(50, -1)),
  cbind(rnorm(50, 0), rnorm(50, 0)),
  cbind(rnorm(50, 1), rnorm(50, 1)),
  cbind(rnorm(50, 2), rnorm(50, 2))))

res <- lapply(c(1:7), function(i) {
  WCSS <- kmeans(data,
                 centers = i, nstart = 10)$tot.withinss
  return(data.frame(i = as.vector(i),
                    WCSS = as.vector(WCSS)))
})
bind_rows(res) |>
  ggplot(aes(x = i, y = WCSS)) +
  geom_point() +
```

```
  geom_line() +
  theme_minimal() +
  scale_x_continuous(breaks = 1:7)
```




```
ggplot(data = data, aes(x = X1, y = X2)) +
  geom_point()
```



We present the cRab algorithm, which utilizes random subsampling of a dataset in order to determine the optimal number of clusters for an unsupervised clustering algorithm.

There will probably be lots of citations here for many articles or even software like R (R Core Team, 2025). (Ben-Hur, Elisseeff, & Guyon, 2001). (Jaeger & Banks, 2023). (Kuhn & Wickham, 2020). (Hvitfeldt & Bodwin, 2025). (Wickham et al., 2019).

1.2 Literature Review

Chapter 2

DETAILS

2.1 Calculated

2.2 Performance on Simulated Data

2.3 Sensitivity

2.4 Known Data

2.5 Performance on penguins dataset

Number of Clusters	Score
2	393.928
3	71.5394
4	7147.85
5	4228.18
6	4700.21
7	4562.12
8	4427.73
9	4246.44
10	3567.82

Chapter 3

COMPUTATIONAL DETAILS

3.1 Matrix Stacking

3.2 Algorithm Construction

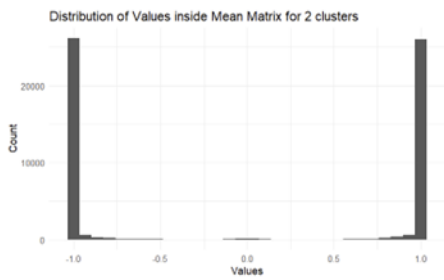
The main principle of our algorithm works off of is the fact that subsamples of a dataset should still capture the overall clustering structure, which is also echoed in the paper from Ben-Hur (2002). This means that given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent. For example, two points that are far away from each other should consistently not be in the same cluster, and two points that are close together should consistently be in the same cluster, even across resamples.

A "clustering assignment matrix" for each resample is then needed, where for points i and j we have a value of 1 in the matrix if they belong to the sample cluster, -1 if not, and 0 if either point was not in the resample.

After we have n matrices, we then reduce the matrices, taking the mean with respect to each index pair, ignoring zeroes. We then receive a final "mean matrix" of size m x m, where m is the number of observations in the dataset.

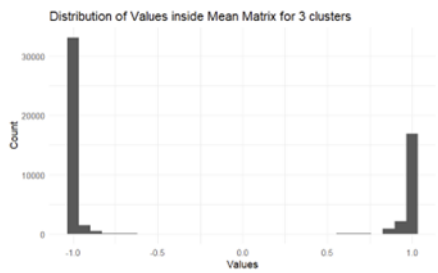
3.3 Score Construction

To illustrate the construction of the final score given this mean matrix, we use the penguins dataset in R with three known species and scaled bill length and flipper lengths.



The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k=2

(replace with code that generates this later)



(replace with code that generates this later)

(Also remove the images folder later)

Figure 2: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k=3

We see that from Figures 1 and 2 that the distribution of values inside the mean matrix changes depending on the number of clusters set beforehand in the K-Means algorithm, albeit slightly. For k = 2, there are a small number of points around the value of 0, which indicates that these specific points are ones who are changing "allegiances" often, meaning that they switch from being "buddies" (being in the same cluster) and to being "rivals", (not being in the same cluster).

Then, to make a final score taking all this into account, we decided to vectorize this matrix, then taking the absolute value and getting the sum of squared distances from 1. This is in order to heavily penalize values that are farther from 1.

3.4 Distribution of Ones and Neg Ones

3.5 Speeed and Memory Scaling

Chapter 4

NULL DISTRIBUTION

4.1 Null Distribution of Scores

It is also interesting information to see what the distribution of scores looks like when there are no clusters in the dataset. This is so that our scores can be adjusted depending on this "null distribution", also allowing us to get a degree of evidence for a given k and other parameters.

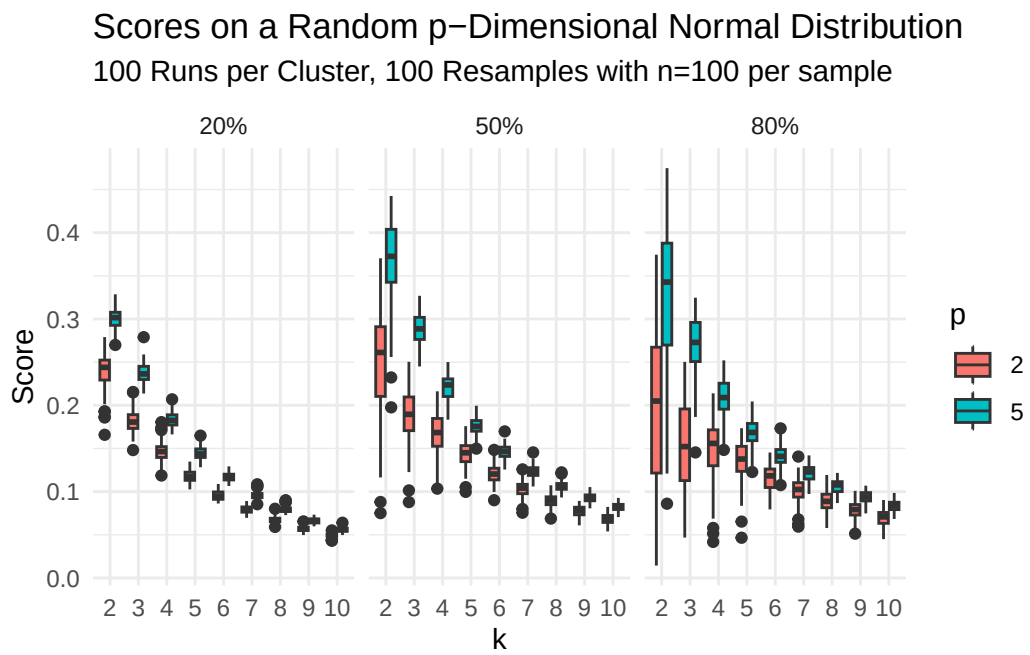


Figure 4.1: Distribution of Scores on a Random p-Dimensional Normal Distribution, Faceted by Resample %

We see in the above figure that the resample proportion has a significant impact on the distribution of scores, where the variability and mean of scores over many simulation runs is dependent on the resample proportion. The scores also tend to be higher, scaling with

p-dimensions, with a decreasing score distribution as k increases. Notably, all these scores look symmetrically distribution around their mean, with respect to each k.

4.2 Null Distribution over Resamples

```
`summarise()` has grouped output by 'factor(k)'. You can override using the
`.groups` argument.
```

Table 4.1: Variances for 20% Resample Calculation

factor(k)	factor(prop_resample)	var
2	0.2	0.1347830
3	0.2	0.1471547
4	0.2	0.1285054
5	0.2	0.1161971
6	0.2	0.1133949

Chapter 5

CONCLUSION

On the penguins dataset, we use 100 resamples and a resample proportion of 80% wih the k-Means algorithm and a given k, using again the scaled bill length and flipper lengths in our dataset.

(i guess this should also go in the analysis portion??)

REFERENCES

Ben-Hur, A., Elisseeff, A., & Guyon, I. (2001). A stability based method for discovering structure in clustered data. *Biocomputing 2002*. Kauai, Hawaii, USA: WORLD SCIENTIFIC. https://doi.org/10.1142/9789812799623_0002

Hvitfeldt, E., & Bodwin, K. (2025). *Tidyclust: A common API to clustering*. https://doi.org/10.32614/CRAN.package.tidyclust

Jaeger, A., & Banks, D. (2023). Cluster analysis: A modern statistical review. *WIREs Computational Statistics*, *15*(3). https://doi.org/10.1002/wics.1597

Kuhn, M., & Wickham, H. (2020). *Tidymodels: A collection of packages for modeling and machine learning using tidyverse principles.* Retrieved from https://www.tidymodels.org

R Core Team. (2025). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from https://www.R-project.org/

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

APPENDICES

Appendix A

RESAMPLE MATRIX

```
resample_function <- function(data = data,
                              formula = ~ .,
                              k = 3,
                              number_of_resamples = 15,
                              proportion_resample = 0.9,
                              starting_seed = 599,
                              algorithm = "kmeans") {
  data <- data |>
    drop_na()
  data$index <- 1:nrow(data)
  results <- list()

  # For loop
  results <- foreach(i = 1:number_of_resamples,
                     .packages = c("tidyclust",
                                   "dplyr",
                                   "tidymodels")) %dopar% {
    # Reproducibility over parallel
    set.seed(starting_seed + i)
    result_matrix <- matrix(0, nrow = nrow(data),
```

```
                          ncol = nrow(data))


random_sample <- data |>
  filter(index %in% sample(index,
                           proportion_resample * nrow(data)))


if (algorithm == "kmeans") {
  cluster_assigned <- k_means(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else if (algorithm == "hier_clust") {
  cluster_assigned <- hier_clust(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else {
  stop("Algorithm is not supported. Please select
       one of (kmeans, hier_clust)")
}



intermediate <- data.frame(random_sample$index,
                           extract_cluster_assignment(
                             cluster_assigned
                             ) |>
                             mutate(.cluster = as.character(
```

```
                                .cluster)
                                ),
                            stringsAsFactors = FALSE)
  colnames(intermediate) <- c("index", "cluster")

  for (c in unique(intermediate$cluster)) {
    idx <- intermediate[intermediate$cluster == c, ]$index

    # Check that the index list is not length 1 for purposes of n > m for combn
    # This also fixes a bug in the
    # previous code where idx was wrongly
    # interpreted as single numerical
    # value as param inside combn
    if (length(idx) > 1) {
      idx <- sort(idx)
      ones <- t(combn(idx, 2))
      result_matrix[cbind(ones[, 1], ones[, 2])] <- 1
    }

    # Note for future self: This calculates the indices, even across y=x line
    # which means we are doing double the work for this part.
    neg_one_idx <- expand.grid(idx, setdiff(random_sample$index, idx))
    result_matrix[cbind(neg_one_idx[, 1], neg_one_idx[, 2])] <- -1
  }
  result_matrix[lower.tri(result_matrix, diag = TRUE)] <- NA
  result_matrix
}
```

```
  return(results)

}
```

Appendix B

MEAN MATRIX

```r
# Got more fancy functions, or figures, or tables,
# or pretty much anything?

mean_matrix <- function(list_of_matrices = list_of_matrices) {
  # Absolute value the entire matrix, for all matrices
  absolute_sum <- lapply(list_of_matrices, abs)

  # Get the sum of absolute values in the final matrix
  absolute_final <- absolute_sum |>
    reduce(`+`)

  # Get the sum of values in the final matrix
  numerator <- list_of_matrices |>
    reduce(`+`)

  # Returns the matrix of zero-removed means.
  # The zero-removed mean is the sum over the (length - the number of zeroes).
  # The length - the number of zeroes is equivalent to the sum over absolute value
  return(numerator / absolute_final)
}
```

Appendix C

METRIC CALCULATION

```r
squared_distance_from_one <- function(mean_matrix = mean_matrix) {
  res_vec <- as.vector(mean_matrix) # Ignore the order of the elements in matrix
  res_vec <- abs(res_vec[!is.na(res_vec)]) # Absolute value and remove NAs
  return(mean((1 - res_vec)^2)) # Take average squared distance from one
}
```

Appendix D

POINT CONTRIBUTION FUNCTION

```
point_contribution <- function(data, mean_matrix) {
  # Given the original data (the dataset inputted into the resample_matrices funct
  # and the mean_matrix, returns each observation's contribution to the cRab Score
  # We don't necessarily require the second arg to be the mean matrix, for example
  # it could be resample matrix j.
  # This is returned as an (index, score) dataframe.

  # Get a list of lists containing index and index "cRab" score
  scores <- lapply(c(1:nrow(data)), function(i) {
    full_vec <- getPointVecHelper(i, mean_matrix)
    singlePointScore <- mean((1 - full_vec)^2)
    return(list(index = i, singlePointScore = singlePointScore))
  })
  final_df <- do.call(rbind.data.frame, scores)
  total_score <- mean(final_df$singlePointScore)

  final_df <- final_df |>
    mutate(singlePointContribution = singlePointScore / total_score)

  return(final_df)
}
```

Appendix E

GET POINT VECTOR HELPER

```
getPointVecHelper <- function(i, mean_matrix) {
  # Given a point i and the mean matrix, should
  # return vector of all values in the mean matrix corresponding to obs i
  # after removing all NAs and then taking absolute value

  row <- mean_matrix[i, ]
  col <- mean_matrix[, i]

  # We don't care about the order, just the values themselves
  res_vec <- c(row, col)
  res_vec <- abs(res_vec[!is.na(res_vec)])
  return(res_vec)

}
```

```
# resample_matrices <- resample_function(penguins,
#                                        number_of_resamples = 40,
#                                        proportion_resample = 0.8,
#                                        starting_seed = 100)
# m_matrix <- mean_matrix(resample_matrices)
#
#
```

```
# d <- point_contribution(penguins, m_matrix)
#
# test <- lapply(resample_matrices,
#        point_contribution,
#        data = penguins)
#
# mean_var_single <- do.call(rbind.data.frame, test) |>
#   group_by(index) |>
#   summarize(mean = mean(singlePointScore),
#             var = sd(singlePointScore)) |>
#   arrange(desc(var))
```